

Argon

Automated Structural Purity Auditor for AI-Generated Code

PURPOSE

Engineering blueprint for an AI coding agent to build Argon end-to-end: data models, API contracts, module functions, and build order.

AUTHOR	Systems Architect & Solo Founder
DATE	September 2026
STACK	Python (FastAPI) · PostgreSQL (Supabase) · Railway
STATUS	Pre-build — Specification Draft

OVERVIEW

1 · What Argon Does

Argon is a GitHub-integrated bot that automatically reviews every Pull Request for **structural bloat** — the kind of overengineered, sprawling code that AI coding agents tend to produce. It posts a scorecard directly onto the PR timeline, and — where a repo opts in — can act as a **hard-blocking GitHub Check** that locks the merge button until the structure clears the bar (Section 4.5). No CLI, no YAML, no manual setup beyond a GitHub OAuth click.

This document breaks the original architecture note down into a build-ready specification: concrete data models, REST contracts, module boundaries, function signatures, and a phased build order — written so an AI coding agent (or a solo developer) can implement it sequentially without having to make architectural judgment calls along the way.

1.1 Guiding Build Principles

- **One module, one job.** Every module below has a single responsibility and a narrow function surface, since Argon should model the discipline it enforces.
- **Stateless compute, stateful ledger.** The analysis engine never persists source code. Only scores and metadata reach Postgres.
- **Build vertically, not horizontally.** Ship Phase 1 (single-repo, JS/TS only, one metric) fully working before adding languages or metrics.
- **Every function in this doc has a defined input/output contract.** An AI agent should implement against the signature, not invent one.
- **Debounced execution by default.** A new push always wins. Any analysis already running for that PR is terminated the instant a newer commit arrives, so Argon never reports on, or blocks a merge based on, a stale HEAD (Section 5.1).

1.2 System Map

GitHub enforces a hard **10-second timeout** on webhook deliveries. AST parsing on a large PR can easily exceed that, so the webhook handler and the analysis engine are fully decoupled: the handler's only job is to verify and enqueue, then return immediately.

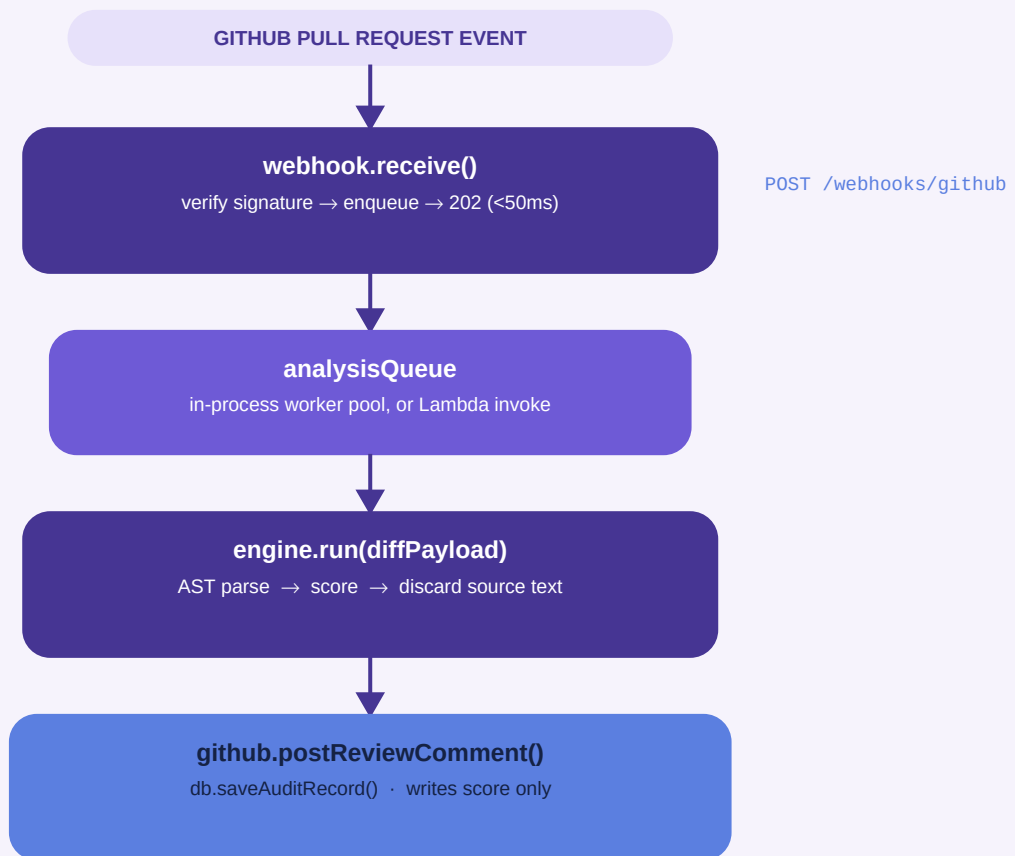


Fig 1. Request lifecycle — webhook returns in milliseconds; the engine runs fully asynchronously off the queue.

INFRASTRUCTURE

2 · Tech Stack & Infrastructure

Two tiers: a lightweight always-on **Web/API tier** on Railway, and a **stateless analysis tier** that runs either in-process (small teams) or forwarded to the customer's own cloud (enterprise).

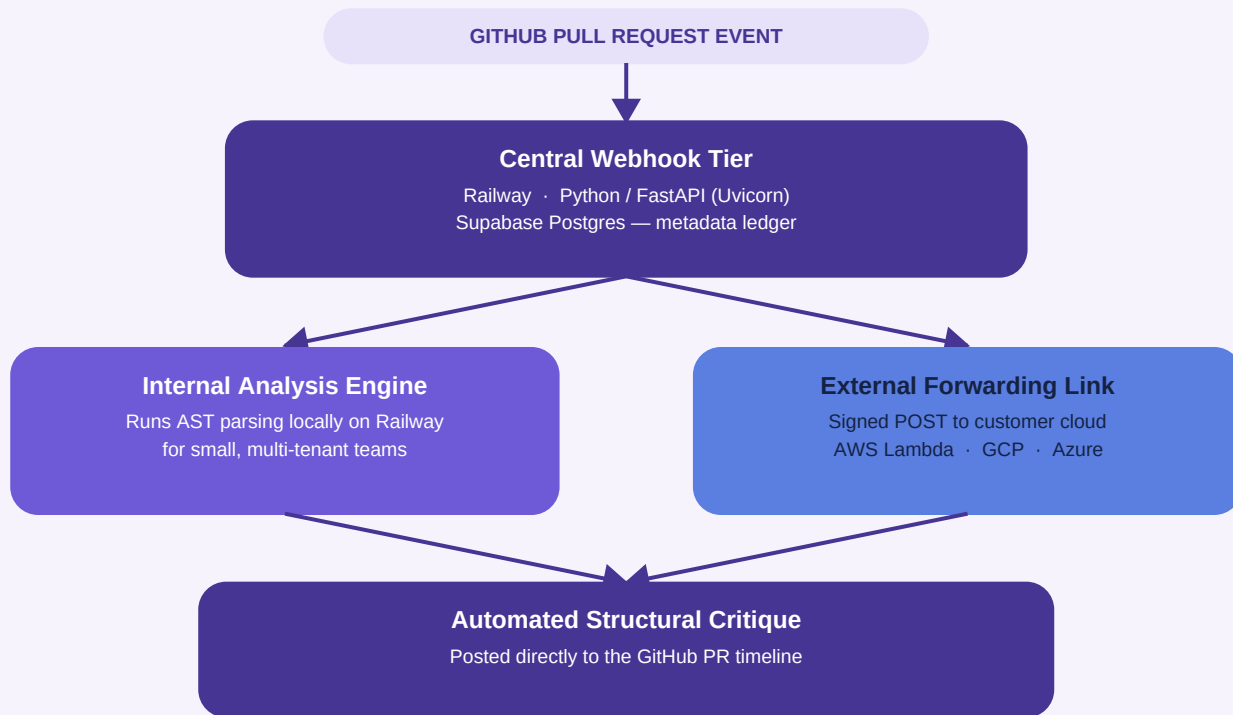


Fig 2. Webhook tier fans out to an internal engine or a customer-owned cloud, then converges back onto the PR.

2.1 Component Decisions

Component	Choice	Why
API server	Python 3.12 + FastAPI (Uvicorn/ASGI)	Async-native, Pydantic validation built in, automatic OpenAPI docs, easy Railway deploy
Database	Supabase Postgres (free tier)	Managed, generous free tier, built-in row-level security for multi-tenant data
Queue	multiprocessing.Process per job, tracked by (repo, PR) (V1) -> Celery + Redis with task revoke (V2 scale)	Python's GIL forces CPU-bound AST work off the main process; a raw Process per job (not a pooled executor) is what makes a stale run cancellable — see 5.2
AST parsing	tree-sitter (py-tree-sitter + JS/TS grammars)	One consistent grammar interface across languages; sets up Phase 3 multi-language support for free

Component	Choice	Why
GitHub integration	GitHub App (not OAuth App) + PyGithub + PyJWT	GitHub Apps get scoped, revocable, per-repo installation tokens; PyJWT signs the App JWT, PyGithub exchanges it for install tokens
Enterprise forwarding	Signed HTTPS POST via httpx (async) to customer worker_url	Keeps proprietary source code inside the customer's cloud boundary
Hosting	Railway (API) — ~\$10–20/mo	Nixpacks auto-detects Python; zero-ops deploy from GitHub push, scales to Fargate/ECS later

2.2 Environment Variables (Phase 1)

```
GITHUB_APP_ID=
GITHUB_APP_PRIVATE_KEY=
GITHUB_WEBHOOK_SECRET=
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
ENVIRONMENT=production
PORT=3000
```

LANGUAGE DECISION — Python end-to-end, no second language

The whole stack — webhook, queue, poster, DB layer, dashboard API, and the analysis engine — is **Python end-to-end**. No Rust, no Node. One runtime, one dependency manager, one deploy pipeline for a solo founder to maintain.

This only works cleanly because AST parsing goes through **tree-sitter** from day one instead of a JS-only parser: py-tree-sitter gives Python a native binding to the same grammars GitHub itself uses, so multi-language support in Phase 3 is just adding grammars — not bolting on a second language runtime.

One real Python-specific constraint to design around: the **GIL** means CPU-bound AST parsing blocks the event loop inside the same process just like it would in Node — so engine/run.py always executes inside a worker process (ProcessPoolExecutor in V1, Celery workers in V2), never inline in the FastAPI process. If a hot loop in ast_counters.py ever needs more raw speed than pure Python gives, reach for hand-written tree-sitter queries or Cython on that one function — not a language rewrite.

2.3 Local Build Gotcha: tree-sitter Grammar Compilation

py-tree-sitter needs the JS and TS grammars (tree-sitter-javascript, tree-sitter-typescript) compiled into a native shared library before it can parse anything.

FIX — Never commit the compiled grammar binary

A .so/.dll compiled on an Apple Silicon dev machine is built for arm64. Deploy that same file to a Linux/AMD64 container on Railway (or any customer AWS/GCP/Azure worker in Section 7) and it crashes on load — the binary is architecture-specific, and committing it to the repo bakes in whatever machine built it.

Compile the grammars as a build step, on the target machine, every time — never track the resulting binary in git. That means a compile script that runs during the Dockerfile build (or the Railway/Nixpacks build phase), so the container always produces its own binary for its own architecture, independent of whatever the developer's laptop is.

```
# scripts/build_grammars.py — run once per build, never at import time
from tree_sitter import Language

Language.build_library(
    "build/grammars.so",          # output path — gitignored, rebuilt every deploy
    [
        "vendor/tree-sitter-javascript",
        "vendor/tree-sitter-typescript/typescript",
    ],
)

# Dockerfile — compiles for THIS container's architecture, not the dev machine's
# FROM python:3.12-slim
# ...
# COPY scripts/build_grammars.py scripts/
# COPY vendor/ vendor/
# RUN python scripts/build_grammars.py
# COPY . .

# .gitignore
# build/*.so
```

DATA MODEL

3 · Data Structures

Two layers of data structures: the **persisted schema** (Postgres, from the original spec, extended with build-ready constraints) and the **Pydantic models** that every module passes between each other. Defining both up front means an AI agent never has to guess a shape.

3.1 Postgres Schema (DDL)

```

-- organizations: one row per GitHub App installation
CREATE TABLE organizations (
  id          VARCHAR PRIMARY KEY,          -- GitHub installation ID
  name        VARCHAR NOT NULL,
  is_enterprise  BOOLEAN DEFAULT FALSE,
  worker_url   TEXT,                        -- NULL unless is_enterprise = true
  created_at   TIMESTAMP DEFAULT now()
);

-- repositories: repos activated under an organization
CREATE TABLE repositories (
  id          VARCHAR PRIMARY KEY,          -- GitHub repo ID
  organization_id  VARCHAR REFERENCES organizations(id) ON DELETE CASCADE,
  repo_name    VARCHAR NOT NULL,
  is_active    BOOLEAN DEFAULT TRUE,
  created_at   TIMESTAMP DEFAULT now()
);

-- audit_history: one row per PR analysis run
CREATE TABLE audit_history (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  repository_id  VARCHAR REFERENCES repositories(id) ON DELETE CASCADE,
  pr_number    INTEGER NOT NULL,
  purity_score  INTEGER CHECK (purity_score BETWEEN 0 AND 100),
  flagged_files INTEGER DEFAULT 0,
  created_at   TIMESTAMP DEFAULT now()
);

CREATE INDEX idx_audit_repo ON audit_history(repository_id);
CREATE INDEX idx_audit_pr   ON audit_history(repository_id, pr_number);

-- active_analyses: tracks in-flight runs so a new push can cancel a stale one.
-- UNIQUE(repository_id, pr_number) enforces "at most one active run per PR"
-- at the database level, not just in application code.
CREATE TABLE active_analyses (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  repository_id  VARCHAR REFERENCES repositories(id) ON DELETE CASCADE,
  pr_number    INTEGER NOT NULL,
  current_head_sha  VARCHAR NOT NULL,
  check_run_id   BIGINT NOT NULL,          -- GitHub Check Run this row's worker updates
  status        VARCHAR NOT NULL DEFAULT 'running', -- running | superseded | completed
  worker_pid     INTEGER,                  -- OS pid backing this run, for cancellation
  started_at    TIMESTAMP DEFAULT now(),
  UNIQUE (repository_id, pr_number)
);

CREATE INDEX idx_active_analyses_pr ON active_analyses(repository_id, pr_number);

```

3.2 Core Pydantic Models

These are the shapes passed between webhook -> queue -> engine -> poster. Every module in Section 5 imports from a single `schemas.py` file — never redefines its own shape.

```
# schemas.py (part 1 of 2)
from pydantic import BaseModel
from typing import Literal, Optional

class ChangedFile(BaseModel):
    filename: str
    status: Literal["added", "modified", "renamed"]
    patch: str # unified diff text
    raw_content_url: str # fetched lazily, never persisted

class AnalysisJob(BaseModel):
    installation_id: str
    repository_id: str
    repo_full_name: str # "owner/repo"
    pr_number: int
    head_sha: str
    changed_files: list[ChangedFile]
    is_enterprise: bool
    worker_url: Optional[str] = None # present only if is_enterprise
    check_run_id: int # provisioned before enqueue, see 5.1 debounce step

class FileFinding(BaseModel):
    filename: str
    line_count: int
    async_boundary_count: int
    cyclomatic_branches: int
    lines_altered: int
    determinate_outputs: int
    relative_import_count: int # cross-imports to sibling files in the same PR, see 4.3
    logic_complexity_rating: float # see Section 4.2 (squared-branch formula)
    flagged: bool # true only for concentration_rule / semantic_path_logic
    flag_reason: Optional[Literal[
        "concentration_rule", "semantic_path_logic"
    ]] = None
```

```
# schemas.py (part 2 of 2)

class AuditResult(BaseModel):
    repository_id: str
    pr_number: int
    head_sha: str          # the exact commit this result was scored against
    check_run_id: int      # GitHub Check Run to complete, see Section 5.5
    purity_score: int      # 0-100, see Section 4.4
    check_conclusion: Literal["success", "failure"] # written to the Check Run, see 4.5
    findings: list[FileFinding]
    flagged_file_count: int
    skipped_file_count: int # dropped by the pre-filter guardrail, see Section 5.3
    fragmentation_flagged: bool # PR-wide signal, see 4.3 – NOT folded into any file's flagged
    summary_markdown: str   # rendered PR comment body

class ActiveAnalysis(BaseModel):
    id: str
    repository_id: str
    pr_number: int
    current_head_sha: str
    check_run_id: int
    status: Literal["running", "superseded", "completed"]
    worker_pid: Optional[int] = None # OS pid backing this run, used by cancel_if_running()
```

CORE LOGIC

4 · Scoring Algorithms & Structural Metrics

Three detection rules plus a combining formula (4.4) that resolves them into a single 0–100 `purity_score`. All three rules exist because each closes a specific loophole an AI coding agent can exploit against a simpler check — see the callouts below each rule.

4.1 Concentration Rule ("Book & Chapter")

Flags a file that is both long *and* juggling multiple independent async responsibilities — the two conditions must **both** be true.

```

LINE_THRESHOLD = 800
ASYNC_BOUNDARY_THRESHOLD = 3

def check_concentration_rule(file: FileFinding) -> bool:
    return (
        file.line_count > LINE_THRESHOLD
        and file.async_boundary_count > ASYNC_BOUNDARY_THRESHOLD
    )

# async_boundary_count is computed by walking the tree-sitter AST and counting distinct:
# - WebSocket / SSE connection handlers
# - independent DB client subscriptions ("hooks")
# - queue consumer registrations (e.g. .on('message', ...))
# De-duplicate by call-site, not by occurrence.
```

4.2 Semantic Path Logic Check

Penalizes circuitous logic that could be simplified — many branches and rewritten lines to reach few distinct outcomes.

FIX — Squared branch count closes the switch-case loophole

The original linear formula ($\text{branches} \times \text{lines} \div \text{outputs}$) is easy to game: wrap a deep logic nest in one giant switch or dictionary lookup, and a matching high output count drags the denominator up enough to cancel the branch penalty out.

Squaring the branch count makes nested branching dominate the score regardless of how many outputs the file claims — a wide flat switch no longer buys a free pass.

```
# Baseline held at 40 even after the squared change – recalibrate only against
# real Phase 1 PR data, and only as a config value, never a hard-coded literal.
SEMANTIC_PATH_FLAG_THRESHOLD = 40

def logic_complexity_rating(
    cyclomatic_branches: int,
    lines_altered: int,
    determinate_outputs: int,  # count of distinct return/output values reachable
) -> float:
    outputs = max(determinate_outputs, 1)  # avoid divide-by-zero

    # Branch count is squared so deep nesting dominates the score even when
    # an agent inflates the output count to shrink the denominator.
    return (cyclomatic_branches ** 2 * lines_altered) / outputs
```

4.3 Structural Fragmentation Rule

Not in the original spec. Catches the file-splitting loophole: an agent dodges the 800-line Concentration Rule by shattering one bloated file into many small, tightly coupled files that import each other heavily to pass state around.

FIX — Import entropy, not file size, is the signal

Individually, each fragment is short and passes the Concentration Rule. What gives it away is **cross-import density** — many small files in the same PR importing directly from one another rather than from stable, pre-existing modules.

`relative_import_count` (Section 3.2) counts only imports using relative paths (`./`, `../`) that resolve to another file changed in the same PR — imports of untouched or third-party modules don't count toward it.

```
FRAGMENTATION_AVG_IMPORT_THRESHOLD = 4  # tune against real PR data
FRAGMENTATION_MIN_FILES = 3             # rule only applies once a PR splits work up

def check_fragmentation_rule(pr_findings: list[FileFinding]) -> bool:
    if len(pr_findings) < FRAGMENTATION_MIN_FILES:
        return False

    total_relative_imports = sum(f.relative_import_count for f in pr_findings)
    avg_relative_imports = total_relative_imports / len(pr_findings)

    return avg_relative_imports > FRAGMENTATION_AVG_IMPORT_THRESHOLD

# relative_import_count itself is produced per-file by count_relative_imports()
# in engine/ast_counters.py (Section 5.4), then this rule looks at the
# PR-wide average across every changed file, not any single file in isolation.
```

4.4 Combining Into `purity_score` (0–100)

Not specified in the original doc — defined here so all three checks resolve to one number.

FIX — Fragmentation must be a flat penalty, never a per-file flag

The obvious way to fold the Fragmentation Rule into this formula is to mark every file in the PR as flagged once the rule trips, so it flows through `flag_penalty_ratio` like the other two rules. That's a bug, not a shortcut: on a 30-file PR where only 10 files actually trip fragmentation, it marks all 30 as flagged — a 100% flag ratio that zeroes the base score out through no fault of the other 20 files.

Fragmentation is a **PR-wide** signal, not a per-file defect, so it can't share the same penalty path as per-file rules without distorting the ratio. It's applied instead as a single flat deduction — `FRAGMENTATION_FLAT_PENALTY` — taken once per PR, completely independent of how many files happened to be part of the fragmented set.

```
FRAGMENTATION_FLAT_PENALTY = 25    # flat, once per PR — never multiplied by file count

def compute_purity_score(
    findings: list[FileFinding],
    fragmentation_triggered: bool,    # output of check_fragmentation_rule(), Section 4.3
) -> int:
    if not findings:
        return 100

    # Only concentration_rule and semantic_path_logic (per-file rules) count toward
    # flag_penalty_ratio. fragmentation_rule must NOT set FileFinding.flagged on every
    # file in the PR — that would let one rule's trigger silently zero out the score
    # of files it never actually flagged.
    flagged_count = sum(1 for f in findings if f.flagged)
    flag_penalty_ratio = flagged_count / len(findings)

    overshoot_sum = sum(
        max(f.logic_complexity_rating - SEMANTIC_PATH_FLAG_THRESHOLD, 0)
        for f in findings
    )
    avg_overshoot = overshoot_sum / len(findings)

    score = 100 - (flag_penalty_ratio * 70) - min(avg_overshoot, 30)

    # Structural Fragmentation Rule (4.3) applies here, once, as a flat deduction.
    if fragmentation_triggered:
        score -= FRAGMENTATION_FLAT_PENALTY

    return round(max(0, min(100, score)))
```

4.5 Hard Gate: `purity_score` → Check Run Conclusion

Argon needs no separate "advisory" vs. "hard-blocking" mode of its own. Reporting through the **GitHub Checks API** instead of just a PR comment is what gives clients the choice: the check's conclusion is always a true success/failure signal, and whether that signal can block a merge is entirely decided by the repo's own branch protection settings.

DESIGN — One signal, GitHub's branch protection decides if it's enforced

A client who wants Argon as a hard gate turns on "Require status checks to pass before merging" for the **Argon Structural Review** check in their branch protection rules. A client who just wants visibility leaves that box unchecked — the same check still posts failure on a bad PR, it just doesn't lock the merge button.

This means Argon's own code never branches on a client-configured "strict mode" flag. It always computes and reports the true conclusion; GitHub owns the enforcement decision. That keeps the module boundary in 5.5 narrow — one function, one job.

```
from typing import Literal

# Config value — a Phase 2 candidate for a per-repo override, but a single
# global constant is correct for V1 (see Section 8).
PURITY_SCORE_FAIL_THRESHOLD = 40

def determine_check_conclusion(purity_score: int) -> Literal["success", "failure"]:
    return "failure" if purity_score < PURITY_SCORE_FAIL_THRESHOLD else "success"

# This conclusion is written onto the GitHub Check Run (github/poster.py, Section 5.5),
# not just rendered into a comment string. GitHub's Checks API is what lets branch
# protection rules key off it — a plain issue-comment API call cannot gate a merge.
```

IMPLEMENTATION

5 · Module Breakdown & Function Signatures

Six modules. Build and test each in isolation, in this order, before wiring them together.

5.1 and 5.2 together implement the **debounced execution pattern**: a new push for a PR that's still being analyzed cancels the stale run outright rather than letting it finish and post against an outdated HEAD.

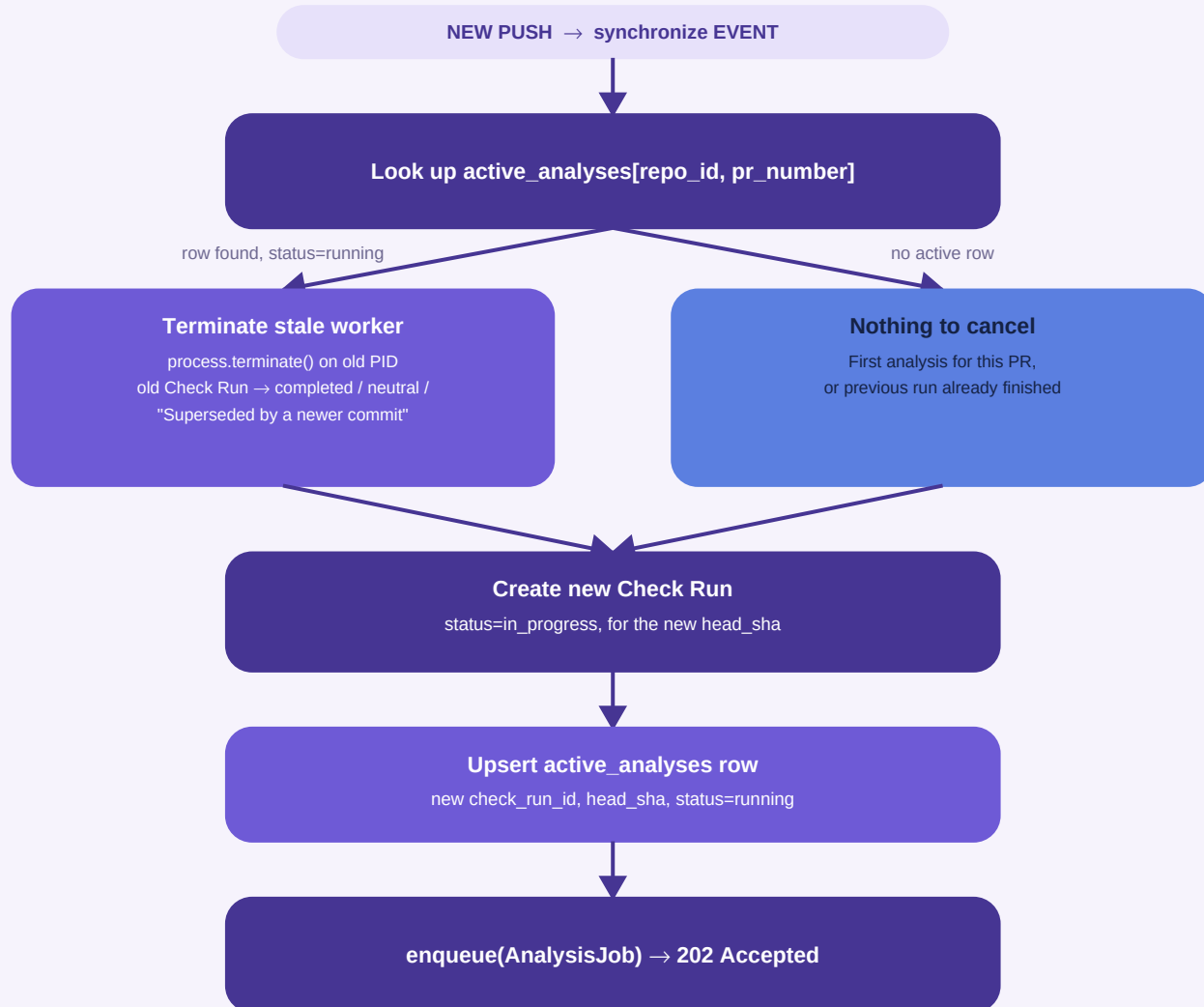


Fig 3. A new commit always wins — the stale worker is terminated and its Check Run marked superseded before the new one is provisioned.

FIX — Provisioning a Check Run adds real network time to the webhook handler

Earlier drafts of this spec promised the webhook handler returns in "<50ms, never touches the network." That no longer holds once the handler is responsible for superseding an old Check Run and creating a new one — those are two real GitHub API round-trips (roughly 200–600ms combined), not local work.

That's still comfortably inside GitHub's 10-second webhook budget, so the fix is honesty, not architecture: the handler is now "typically well under a second, dominated by two GitHub API calls," not sub-50ms. AST parsing itself still never runs inline — that stays entirely on the worker process from Section 5.2.

5.1 webhook/receiver.py

Receives and validates GitHub webhook events, and runs the debounce step before enqueueing. Nothing in this module parses AST or blocks past the GitHub API calls described above.

Function / Route	Responsibility
<code>verify_signature(payload: bytes, signature: str, secret: str) -> bool</code>	HMAC-SHA256 check against GITHUB_WEBHOOK_SECRET (hmac + hashlib, stdlib). Reject with 401 on mismatch. Runs before anything else touches the payload.
<code>parse_pull_request_event(body: dict) -> AnalysisJob None</code>	Returns None (and 202-acks) for any action other than opened/synchronize/reopened. check_run_id is not set yet at this point.
<code>async def debounce_and_provision_check(job: AnalysisJob) -> AnalysisJob</code>	<code>db.get_active_analysis(job.repository_id, job.pr_number)</code> -> if a running row exists: <code>queue.cancel_if_running(key)</code> to terminate the stale worker, then <code>poster.supersede_check_run(...)</code> marking the old Check Run completed/neutral/"Superseded by a newer commit. Recalculating..." -> always <code>poster.create_check_run(...)</code> for the new head_sha -> <code>db.upsert_active_analysis(...)</code> -> returns job with check_run_id set.
<code>@app.post("/webhooks/github")</code>	FastAPI route. verify -> parse -> debounce_and_provision_check -> <code>queue.enqueue(job)</code> -> return 202. See the network-time callout above for why this is no longer sub-50ms.

5.2 queue/analysis_queue.py

Runs each job as its own OS process (not a pooled executor) so a stale run can be cancelled outright when a newer commit supersedes it. CPU-bound AST work always leaves the FastAPI process — Python's GIL means it would block the event loop otherwise.

Function / Route	Responsibility
<code>enqueue(job: AnalysisJob) -> None</code>	Starts a fresh multiprocessing.Process running <code>engine.run(job)</code> , keyed by <code>(repository_id, pr_number)</code> in an in-memory registry mirroring <code>active_analyses</code> . Returns immediately — the caller (5.1) never awaits this.
<code>cancel_if_running(key: tuple[str, int]) -> bool</code>	Looks up the registry for that <code>(repository_id, pr_number)</code> ; if a process is alive, calls <code>.terminate()</code> and <code>.join(timeout=2)</code> . Safe specifically because <code>engine.run()</code> writes nothing until it completes (Section 1.1's stateless-compute principle) — there's no partial state left behind by killing it mid-parse.

5.3 engine/run.py

The stateless analysis core. Never writes source text to disk or DB. Pre-filters noise before spending any CPU on AST parsing. Always executes inside the worker process started by 5.2, never inline in the API process.

Function / Route	Responsibility
<code>should_skip_file(filename: str, content_preview: str) -> bool</code>	Global pre-filtering guardrail, runs first for every changed file. True if the extension matches a lockfile/manifest/build-output blocklist (e.g. <code>package-lock.json</code> , <code>*.generated.*</code> , Prisma schema output), or if <code>content_preview</code> 's first 5 lines contain an <code>@generated</code> tag or a known compiler header.
<code>async def run(job: AnalysisJob) -> AuditResult</code>	Orchestrates: <code>should_skip_file()</code> per file -> fetch diffs for the rest -> parse AST per file (tree-sitter) -> score each file (4.1, 4.2) -> <code>check_fragmentation_rule()</code> across the PR -> <code>compute_purity_score(findings, fragmentation_triggered)</code> -> <code>determine_check_conclusion(purity_score)</code> (4.5) -> build <code>AuditResult</code> (carrying <code>job.head_sha</code> and <code>job.check_run_id</code>) -> explicitly drop file contents from memory before returning.
<code>analyze_file(file: ChangedFile) -> FileFinding</code>	Parses one file's AST, applies 4.1 and 4.2, returns a <code>FileFinding</code> (<code>relative_import_count</code> included for 4.3's PR-wide check — never sets this file's own <code>flagged/flag_reason</code> from fragmentation).
<code>build_summary_markdown(result: AuditResult) -> str</code>	Renders the PR comment body (see 5.5 template).

5.4 engine/ast_counters.py

Low-level tree-sitter walkers shared by `analyze_file()`.

Function / Route	Responsibility
<code>count_async_boundaries(tree: Node) -> int</code>	Walks the tree-sitter tree; counts distinct WS/DB-hook/queue-consumer call sites.
<code>count_cyclomatic_branches(tree: Node) -> int</code>	Standard McCabe complexity walk (if/else, switch cases, &&/ , ternaries, loops).
<code>count_determinate_outputs(tree: Node) -> int</code>	Counts distinct literal/variable values reachable via return statements.
<code>count_relative_imports(tree: Node, pr_file_set: set[str]) -> int</code>	Counts import/require specifiers starting with <code>./</code> or <code>../</code> that resolve to a filename also present in <code>pr_file_set</code> — i.e. imports of sibling files changed in the same PR. Feeds the Structural Fragmentation Rule (4.3).

5.5 github/poster.py

Posts results back onto the PR: a Check Run for gating, a timeline comment for human-readable detail. Both use an installation access token via PyGithub/PyJWT, never a long-lived PAT.

Function / Route	Responsibility
<code>async def create_check_run(installation_id: str, repo_full_name: str, head_sha: str) -> int</code>	Creates a GitHub Check Run named "Argon Structural Review" with status= <code>in_progress</code> for head_sha. Returns the new check_run_id.
<code>async def supersede_check_run(installation_id: str, repo_full_name: str, check_run_id: int) -> None</code>	Patches an existing Check Run to status= <code>completed</code> , conclusion= <code>neutral</code> , output.title="Superseded by a newer commit. Recalculating...". Called on the old check_run_id before a new one is created.
<code>async def complete_check_run(installation_id: str, repo_full_name: str, result: AuditResult) -> None</code>	Patches result.check_run_id to status= <code>completed</code> , conclusion=result.check_conclusion (4.5), with output.summary built from purity_score and flagged_file_count. This is the call that actually locks or unlocks the merge button, via the client's own branch protection settings.
<code>async def post_review_comment(installation_id: str, repo_full_name: str, pr_number: int, body: str) -> None</code>	Upserts — edits Argon's own prior comment on the same PR instead of duplicating. Runs alongside complete_check_run(), not instead of it.
<code>async def find_existing_argon_comment(...) -> int None</code>	Looks for a comment authored by the Argon bot user to update in place.

5.6 db/repository.py

All Postgres access — the only module allowed to import the DB client.

Function / Route	Responsibility
<code>async def upsert_organization(org: OrganizationRow) -> None</code>	
<code>async def upsert_repository(repo: RepositoryRow) -> None</code>	
<code>async def save_audit_record(result: AuditResult) -> None</code>	Writes only score + counts — never file contents or diffs.
<code>async def get_score_history(repository_id: str, limit: int = 30) -> list[AuditHistoryRow]</code>	Powers the dashboard trend chart.
<code>async def get_active_analysis(repository_id: str, pr_number: int) -> ActiveAnalysis None</code>	Looked up first in <code>debounce_and_provision_check()</code> (5.1) to decide whether a stale run needs cancelling.
<code>async def upsert_active_analysis(analysis: ActiveAnalysis) -> None</code>	Writes on <code>UNIQUE(repository_id, pr_number)</code> — a new row for this PR always replaces the old one, matching the DDL constraint in Section 3.1.
<code>async def clear_active_analysis(repository_id: str, pr_number: int) -> None</code>	Called once <code>run()</code> (5.3) completes successfully and the final Check Run + comment are posted — marks the row <code>status=completed</code> or deletes it, so the next push finds no stale run to cancel.

API CONTRACT

6 · Dashboard REST API

Not present in the original spec but required to render the dashboard UI referenced in Section 3. All routes require a Supabase-issued session JWT except the webhook route.

Method & Path	Purpose	Response shape
GET /api/organizations/me	Current user's org + plan tier	OrganizationRow
GET /api/repositories	List active repos for the org	RepositoryRow[]
POST /api/repositories/:id/toggle	Activate/deactivate auditing for a repo	{ is_active: boolean }
GET /api/repositories/:id/history?limit=30	Score trend for dashboard chart	AuditHistoryRow[]
GET /api/repositories/:id/latest	Most recent audit summary	AuditResult (score + counts only)
POST /api/organizations/me/worker-url	Set enterprise worker_url (enterprise plan only)	{ worker_url: string }
POST /webhooks/github	GitHub App webhook receiver (Section 5.1)	202 Accepted (async)

6.1 Example PR Comment Output

What github/poster.py renders onto the PR timeline — this is Argon's entire user-facing product surface for a developer.

```
### Argon Structural Review -- Purity Score: 78/100

| File | Issue | Rule |
|---|---|---|
| src/orders/handler.ts | 940 lines, 4 async boundaries | Concentration Rule |
| src/utils/validate.ts | Logic complexity 52 (threshold 40) | Semantic Path Logic |

2 of 14 changed files flagged. No blocking gate applied – see .argonignore to suppress.
```

6.2 Check Run vs. PR Comment

Argon posts two things per analysis, and they serve different audiences. The **Check Run** (5.5, 4.5) carries the true/false conclusion that GitHub's branch protection can key off — that's the gating surface. The **PR comment** above is the human-readable detail: which files, which rule, what the numbers were. A repo with branch protection off still gets the comment; it just never blocks a merge.

SECURITY

7 · Security & Enterprise Routing

For enterprise customers, Argon never receives raw source code — the diff payload is forwarded, signed, directly to the customer's own cloud function, which returns only a list[FileFinding] array back over HTTPS.

7.1 Enterprise Forward Contract

```
# engine/forward.py
import httpx

async def forward_to_customer_worker(
    job: AnalysisJob,
    worker_url: str,
) -> list[FileFinding]:
    signature = sign_payload(job, PLATFORM_SIGNING_KEY)

    async with httpx.AsyncClient() as client:
        res = await client.post(
            worker_url,
            json=job.model_dump(),
            headers={
                "Content-Type": "application/json",
                "X-Argon-Signature": signature,
            },
        )
    if res.is_error:
        raise AnalysisForwardError(res.status_code)

    return [FileFinding(**f) for f in res.json()]
# customer worker runs the SAME open-sourced scoring logic
```

7.2 One-Click Cloud Deploy Targets

Cloud	Mechanism	Deploy artifact
AWS	Deep-linked CloudFormation Quick-Create URL	argon-worker-lambda.template.yaml (Python 3.12 runtime)
GCP	"Deploy to Google Cloud" button (Cloud Run button spec)	cloudbuild.yaml + Dockerfile (python:3.12-slim base)
Azure	ARM template deploy-to-Azure link	azuredeploy.json (Python Function App)

7.3 Human Guardrails

- .argonignore at repo root — glob patterns excluded from analysis entirely.
- Inline // argon-skip comment — suppresses flags for the next AST node only.

- Argon can hard-block a merge (Section 4.5) once a client turns on "Require status checks to pass" for the Argon check — until then, or for clients who never turn it on, it only comments and reports. Either way, `.argonignore` and `argon-skip` stay available as an escape hatch.

ROADMAP

8 · Recommended Build Order

Each phase should be fully working — testable end-to-end on a real repo — before starting the next.

PHASE 1 — MVP (1 repo, JS/TS only)

- `schemas.py` (Pydantic models, including `ActiveAnalysis`) + Postgres schema (Section 3, including `active_analyses`)
- `webhook/receiver.py` + `queue/analysis_queue.py` — HMAC verify -> `debounce_and_provision_check` -> `enqueue` -> 202 (typically well under a second, dominated by two GitHub API calls)
- Debounced execution: `cancel_if_running()` + `supersede_check_run()` wired end-to-end before any other module depends on them
- `engine/run.py` pre-filter guardrail (`should_skip_file`) before any AST parsing
- `engine/ast_counters.py` + `analyze_file()` for `.ts/.js` grammars only, including `count_relative_imports`
- All three scoring rules live: Concentration, squared Semantic Path Logic, Structural Fragmentation (flat PR-level penalty, never a per-file flag)
- `determine_check_conclusion()` (4.5) + `create_check_run()/complete_check_run()` in `poster.py` — the hard-gate path is core to V1, not deferred
- Manual test: open a PR, confirm a Check Run and comment both appear; push a second commit mid-analysis and confirm the first Check Run is marked superseded, not left dangling

PHASE 2 — Multi-repo SaaS dashboard

- GitHub App installation flow + `organizations/repositories` tables live
- `queue/analysis_queue.py` scaled to Celery + Redis — cancellation becomes `celery_app.control.revoke(task_id, terminate=True)` in place of `process.terminate()`
- Dashboard REST API (Section 6) + score history chart
- Comment upsert (edit instead of duplicate) in `poster.py`
- Consider a per-repo override for `PURITY_SCORE_FAIL_THRESHOLD` (4.5) instead of the single V1 global constant

PHASE 3 — Enterprise, guardrails & performance

- `engine/forward.py` + one-click cloud deploy templates (Python runtimes)
- `.argonignore` and `// argon-skip` support
- Multi-language support beyond JS/TS — add more tree-sitter grammars, same Python pipeline
- Profile `ast_counters.py` under real load; if a specific hot loop is the bottleneck, optimize that one function (tree-sitter queries or Cython) rather than reaching for a second language

This specification extends the original Argon architecture note (Sept 2026) with concrete data contracts, function signatures, a phased build order, and fixes for six exploitable or correctness gaps: unbounded webhook blocking, the squash-the-denominator complexity loophole, file-splitting to dodge the line-count rule, a per-file flag path that let the Fragmentation Rule silently zero out unrelated files' scores, a stale-HEAD race condition

where a fast follow-up commit could be scored and gated against outdated code, and cross-architecture crashes from committing a compiled tree-sitter binary. Constants marked as tunable (Sections 4.2, 4.3, 4.5) should be treated as config values, not hard-coded literals, so they can be calibrated once Phase 1 has real PR data to test against.